

Blinded-V5: the LGI-based compute module

design rationale, construction, parameters, and measurements

local-mixing project

2026-09-04

Blinded-V5 is a drop-in alternative to the drip `route_fire` compute: it takes a reversible circuit A on n wires (the sliced sandwich) and emits an equivalent circuit A' on $2n$ wires (data $0..n$, band $n..2n$) whose body is a shuffled cloud of long masking identities with A 's gates threaded through it. Everything else in the 5-step gadgetization is unchanged. Source: `src/preprocessing/blinded.v5.rs`. The Markdown twin of this document is `BLINDED-V5.LGI-DESIGN.md`.

1 Why this design — the rationale

The compute stage must move the circuit's intermediate state as far as possible from the plaintext computation A , while keeping A' exactly equivalent to A on the data wires. The design is a chain of forced moves.

(a) **Long identities are the only thing that moves the state substantially.** Across the HD (Hamming-distance) and affine heatmaps, short local edits barely displace the state: the ridge of recoverable structure only recedes when the masking identity woven through a wire is *long* — comparable to the circuit itself. Short masks are peeled off locally; a long identity has no local seam to peel.

(b) **The only long identity we know that actually moves the state is the commutative structure of an identity with a single active wire.** Take one data (“active”) wire w and a set of control wires; a sequence of gates that all target w and read only the controls is an identity iff the per-gate increments XOR to zero, and — crucially — *all of its gates commute* (same target, disjoint from the controls' evolution). Commutativity is what lets the identity be arbitrarily long and be reordered or interleaved freely without changing its value. A single active wire is what makes the whole run a clean identity on that wire.

(c) **We entangle identities on different active wires by separating the control wires from the active wires, then sharing the same control wires across many identities at once — again using commutativity.** Each data wire w gets its own family of single-active-wire identities, but they all draw their controls from a common **band** ($n..2n$). Because every identity targets a data wire and reads only band wires, identities on *different* active wires also commute with each other. The scaffold is therefore one large commuting cloud: many long identities, on many active wires, sharing controls, interleaved arbitrarily. The shared controls are what “entangle” the identities — a band wire's value participates in the masks of many data wires simultaneously.

(d) **After entangling by commutation, we add random updates of the control wires from the active wires.** A pure commuting cloud has a weakness: an adversary can commute the identities back apart, because nothing couples the control wires to time or to the data. So we sprinkle in gates that *update* band wires, half controlled by live data wires ($b \oplus = \text{data} \wedge \text{aux}$) and half by other band wires ($b \oplus = \text{aux} \wedge \text{aux}$). These updates serve two purposes at once:

1. they **re-randomize** the control wires (fresh band entropy throughout the run, not just at the seed); and
2. they **break naive commutation-back**: once a band wire has been updated mid-run, the identities that read it before and after the update no longer commute past it, so the cloud cannot be trivially un-shuffled — *without ruining correctness*, because each identity is repaired across the update (§2, REPAIR) or closed before it (STRADDLE).

(e) **Finally we build one identity longer than the entire input circuit and embed A inside it, using a hidden unmasking of the control wires.** The scaffold above is, in effect, a single enormous LGI spread over all wires. We thread A ’s gates through it so that each gate fires *from inside the mask*: at each gate we momentarily and reversibly **unmask** its operands into a linear combination of band wires (never the bare operand), fire the gate as an expansion over those masked wires, and re-mask. The circuit is embedded inside a single very long identity: **when an A -gate fires, its operands are read only as linear combinations of band wires — never bare** (measured 0.000% bare reads, §4). This is a fire-time property of the operands; the residual plaintext a *global* adversary can still recover is the public I/O fringe of §4.2 (input x and the delivered $C(x)$), not an artifact of the embedding.

2 The construction

A has n wires and m gates. A' has $2n$ wires. Let u_w be the number of times wire w is used (target or control) in A .

Step 0 — band seed (separate module). Each band wire is seeded $x_i \wedge \neg x_j$ from the honest active inputs by `seed.band`, pipelined in front of the compute. The compute does **not** seed the band (module 0 does); it reads the band as LGI controls and re-randomises it in place via the Step-3 updates.

Step 1 — lgi scaffold. For each data wire w , build $u_w + 1$ identities. Each identity is a set of K band wires (“the LGI’s controls”) realised as $\lfloor K/2 \rfloor$ **disjoint g57 pairs** $\text{g57}(w, r_{2i}, r_{2i+1})$, where $\text{g57}(w, x, y) = w \oplus 1 \oplus (\neg x \wedge y)$ (comp = 1). Applying the pair once and removing it once is the identity; the mask it carries between apply and remove is a disjoint degree-2 form $\bigoplus_i (1 \oplus \neg r_{2i} \wedge r_{2i+1})$ — the optimal sparse-mask shape. All $\approx K(3m + n)$ scaffold gates commute (data targets, band controls).

Why disjoint pairs, not a K -cycle: a g57 gate and its reverse **linearise**, $\text{g57}(w, r_1, r_2) \oplus \text{g57}(w, r_2, r_1) = w \oplus r_1 \oplus r_2$ (AND monomials are symmetric and do not linearise). Under the read’s pair-completion, a full K -cycle telescopes to $\bigoplus (r_i \oplus r_{i+1}) = 0$ around the cycle — the operand would go **bare**. Disjoint pairs linearise to a nonzero $\bigoplus_i (r_{2i} \oplus r_{2i+1})$, so the operand is always masked. (Measured: cycle form $\rightarrow$ 6.3% bare reads; disjoint pairs $\rightarrow$ 0.13%; with on-demand top-up $\rightarrow$ 0.000%.)

Step 2 — ordering. Emit the identities with at most `max_open` open per wire (a rolling window), interleaving the per-wire Open/Close event streams randomly.

Step 3 — control-wire updates (rerand), two kinds. Splice band updates $b \oplus = \text{lit} \wedge \text{lit}$ at sorted positions, each kind’s total split half data-control / half band-only:

- **straddle** (`rerand_level`): before the update, *close* the open identities that read b so none carries a stale b across it. Heavy straddle doses thin the masking (empirical knee ≈ 1024).
- **repair** (`rerand_repair`): re-derive each straddling identity across the update — emit its b -reading `g57` with old- b (removing its contribution), the update, then again with new- b (re-adding it). Old- b cancels, new- b re-derives; the identity *stays open*, so REPAIR adds band turnover with *no* thinning. Data controls draw only from live-data wires (`0..active`), which in the sandwich is the honest low half — never the dead zero slice.

Step 4 — embed A with the masked (hidden-unmasking) read. Interleave A ’s gates evenly through the scaffold. At each gate, for every control:

1. **linearise** — emit the reverse `g57` of every net-open mask so the wire carries $\text{operand} \oplus \rho$, with ρ a linear XOR of band wires; top up on-demand with fresh complete linear pairs until the mask spans $\geq \text{max_open}$ pairs and ρ is never empty (guarantees no bare read);
2. **fire** — realise $c \oplus = \text{comp} \oplus \text{lit}(a) \wedge \text{lit}(b)$ as the expansion $(a' \oplus \rho_a)(b' \oplus \rho_b)$ using only the masked control wires and band wires, never the bare a , b , or $a \oplus b$ (0/1/2-control gates and all polarities handled; the 0-control fire is $\text{comp} \oplus \text{AND}(\emptyset) = \neg \text{comp}$);
3. **de-linearise** — undo (1) in reverse.

Step 5 — no discipline pass. The surrounding pipeline (junk-guard zero-slice guards as modules 1 and 5) is unchanged.

Correctness is verified exhaustively over all 2^n inputs $\times$ many band settings for $n \leq 6$, $K \in 2..5$, all `max_open` and both rerand kinds (`scratchpad/v6`, 594 gadgets, 0 mismatches), and end-to-end in `gen_sandwich_gadget` (forward + reverse-honesty sample-verify PASS). $K \geq 2$ is required (a `g57` 1-cycle is a degenerate constant flip).

3 Parameters and why

knob	prod.	meaning	why this value
K	2	band wires per LGI = per-LGI mask width ($\lfloor K/2 \rfloor$ disjoint pairs; $ \rho \approx \text{max_open} \cdot K$) — <i>not</i> the identity’s temporal length	affine- and deg-2-neutral across K (§4); smallest is best. Size grows $\sim$ linearly in K , and the read cost is <i>quadratic in</i> K , and the read cost is <i>quadratic in</i> $ \rho \approx \text{max_open} \cdot K$, so large K explodes: $K2 \approx 654\text{k g}$, $K4 \approx 1.9\text{M}$, $K8 \approx 5.8\text{M}$, $K16 \approx 18\text{M}$ (unusable) . Odd K wastes a wire ($K3 \equiv K2$).
max_open	3	rolling cap on open LGIs per wire; also the on-demand read-mask width floor	wider ρ = more local hiding, but read cost is quadratic in $ \rho $. 3 is the knee.
rerand_level	1000	STRADDLE updates (close straddling masks)	at the safe side of the ≈ 1024 thinning knee.
rerand_repair	3000	REPAIR updates (re-derive straddling masks; no thinning)	stacks band turnover on top of straddle at zero masking cost (grows size only).
rerand split	half/half	data-control vs band-only, per kind	data controls read live data (breaks commutation-back and couples band to data); band-only controls add pure band entropy.

K sets each LGI’s **mask width** (control wires per LGI), *not* its temporal length — the identity’s circuit-spanning length is fixed by the scaffold (§1e), which is what makes it “one identity longer than the whole circuit.” K is exposed as `--bv5-k` in `gss_mix.sh`.

4 Measurements

4.1 Global affine / degree-2 ridge (hiding of source- C segments)

Adversary: least-squares recovery of each source- C wire-segment from a bounded set of gadget wires (`hmap_affine --degree 1/2`, ridge read with `reports/plot_hmap_ridge.py`). meanH: 0 = recoverable, 0.5 = fully hidden.

- meanH ≈ 0.498 for every config $\Rightarrow C$ -segments hidden overall.
- Affine ridge depth ≈ 0.055 , degree-2 ≈ 0.07 (quadratic reaches ~ 0.02 deeper into the low-degree I/O fringe). Both are **flat across** $K = 2, 4, 8, 16$ — a wider per-LGI mask buys no additional hiding, which is why the smallest K wins.

(This sweeps K = the mask *width*; the identity’s temporal *length* — §1(a) — is held fixed circuit-spanning by the scaffold in every config, so it is not the axis being varied here. The length claim of §1(a) rests on the HD/affine heatmap observation that short masks recede locally and long ones do not, which is what motivated the circuit-spanning scaffold.)

4.2 Linearly-exposed source- C segments — the public-I/O fringe

$\approx 8\text{--}10\%$ of source- C segments are substantially linearly exposed ($\text{min-}H < 0.25$ over cuts), flat across K and n . They are *not scattered*: they are two smooth monotone ramps at the diagonal **endpoints** — the input side (segments affine in the *known* input x , which the adversary computes anyway) and the output side (converging on the delivered $C(x)$). The interior **78% of segments are fully hidden** ($H \approx 0.5$). This fringe is **unavoidable public I/O**, not a leak; the boundary-cushion knobs (`extra_lgis`, `a_margin`) measured no effect and are off by default. Fringe width $\approx 2n_C$ gates = C ’s degree-growth depth.

4.3 Rerand: repair vs straddle ($n_A = 256$, affine deg-1)

Same total dose, straddle vs repair:

straddle	repair	gadget	meanH	affDepth
0	0	670k	0.4979	0.0550
1000	3000	668k	0.4959	0.0514
8000	0	607k	0.4878	0.0681
0	8000	780k	0.4985	0.0549

Heavy STRADDLE thins (meanH $\downarrow$, ridge $\uparrow$, gadget *shrinks* as masks close); heavy REPAIR does not (both = baseline, gadget *grows* from the pre/post pairs). The production 1000 + 3000 sits at the baseline floor.

4.4 Through the GSS pipeline ($K = 2$ vs $K = 4$)

Pending — from the `gss_mix.sh --gadgetization-mode blinded-v5` runs ($n = 128$, half-size $|C| = |D| = 3000$; phase-A grow-to- $2\times$ then hold ≈ 10 effs; snapshot at 2 effs). To be appended: affine ridge and count of linearly-exposed C -segments at the 2-eff snapshot and at the end of the pipeline, for $K = 2$ and $K = 4$, with the gadget / phase-A / split / crossing sizes.

5 Tradeoffs and the current choices

- **Mask width K vs size.** Larger K widens each per-LGI mask ($|\rho| \approx \text{max_open} \cdot K$) — it does *not* lengthen the identity’s temporal reach, which the scaffold fixes circuit-spanning (§1e). Empirically the hiding statistics (affine and degree-2 ridge, meanH, exposed-fringe fraction) are *flat in K* : widening the local mask buys no additional hiding, while size grows $\sim$ linearly and the masked-read cost grows *quadratically* in $|\rho| \approx \text{max_open} \cdot K$. K16 (the historical choice) produces an $\sim 18\text{M}$ -gate gadget with no measured hiding gain over K2’s $\sim 650\text{k}$. **We choose K2**; K4 is available only when a wider read-mask ($\rho \approx 13$ vs 7) is wanted. (The temporal-length lever of §1(a) is not K — it is the scaffold spanning the whole circuit, on by construction.)
- **Band turnover vs size (straddle vs repair).** More control-wire updates = more band re-randomisation and stronger resistance to commutation-back. But STRADDLE updates *thin* the masking past ≈ 1024 (they close masks), degrading the ridge; REPAIR updates preserve masking but *grow* the gadget. We take **1000 straddle** (at the safe knee) + **3000 repair** (turnover without thinning), balancing band entropy against size.

- **Read-mask width (`max_open`) vs read cost.** A wider ρ spreads each operand over more band wires locally, but the fire expands to $O(|\rho|^2)$ terms. `max_open = 3` is the knee.
- **The residual fringe is I/O, not a knob.** The $\sim 10\%$ exposed segments are the public input/output boundary; no parameter removes them (they are what the adversary already knows), so we do not pay size to chase them.

6 Running it

```
# one arm of the pipeline, LGI compute, K=2, half-size n=128 sandwich
scripts/gss_mix.sh -n 128 --mcd 3000 \
  --gadgetization-mode blinded-v5 --bv5-k 2 \
  --expand 2 --hold 10 -o RUNDIR
# (K=4: --bv5-k 4). fmix stages require FROZEN_DB_DIR (+ FROZEN_CURATED_DIR).

# just the gadget (stage 2), or the standalone driver:
scripts/gss_mix.sh -n 128 --mcd 3000 --gadgetization-mode blinded-v5 \
  --bv5-k 2 --stop-after 2 -o RUNDIR
blinded_v5_gadgetize src.mpmct1 out.mpmct1 <K> 0 <seed> \
  <straddle> 3 <active> 2 0 <repair>
```